



O Senhor dos
Bytes:
Uma Jornada
Épica em Java

Caroline Teixeira



Linguagem Java

Java é uma linguagem poderosa e versátil, forjada para construir aplicações robustas e escaláveis. Como uma lâmina élfica, ela guia desenvolvedores por terras desafiadoras de código com precisão e força.

Neste e-book, embarcaremos em uma jornada épica pelos fundamentos e técnicas avançadas de Java, com exemplos práticos que iluminarão seu caminho. Prepare-se para forjar suas habilidades e conquistar os desafios da Terra dos Bytes!

01

A Forja de Java – Seus Primeiros Passos

A Forja de Java – Seus Primeiros Passos



Por que aprender Java?

Java é uma linguagem versátil, forjada para criar desde aplicativos até jogos. Assim como uma espada élfica de Frodo, ela é confiável e funciona em qualquer sistema operacional, sendo ideal para iniciantes.

Primeiros passos

1. Instale o Java Development Kit (JDK) no site da Oracle.
2. Use uma IDE como IntelliJ IDEA, Eclipse ou VSCode.
3. Escreva seu primeiro programa.

A Forja de Java – Seus Primeiros Passos



Exemplo: Programa "Olá, Mundo!"

Imagine que você quer saudar os habitantes de um novo reino ao iniciar sua jornada. Abra a sua IDE favorita e digite:

```
OlaMundo

public class OlaMundo {
    public static void main(String[] args) {
        System.out.println("Olá, Mundo");
    }
}
```

A Forja de Java – Seus Primeiros Passos



Exemplo: Programa "Olá, Mundo!"

```
OlaMundo

public class OlaMundo {
    public static void main(String[] args) {
        System.out.println("Olá, Mundo");
    }
}
```

- **public class OlaMundo:** Define uma classe chamada "OlaMundo", que é o ponto de partida do programa.
- **main:** O método principal onde a execução começa.
- **System.out.println:** Comanda a exibição de texto no console.

Dica: Salve o arquivo como OlaMundo.java (o nome deve ser igual ao da classe) e execute com java OlaMundo.

02

Baús de Dados – Armazenando Variáveis

Baús de Dados – Armazenando Variáveis



Variáveis

Variáveis são como baús mágicos onde você guarda informações importantes, como números ou textos. Em Java, você precisa definir o tipo de dado que cada baú pode conter.

Tipos comuns de dados:

- **int:** Para números inteiros, como 42.
- **double:** Para números com casas decimais, como 3.14.
- **String:** Para textos, como "Ana".
- **boolean:** Para dados com dois valores, como Verdadeiro ou Falso.

Baús de Dados – Armazenando Variáveis



Exemplo: Calculando o Preço de um Produto

Imagine que você é um mercador e quer calcular o preço com desconto de um item:

```
CalculoDesconto

import java.util.Scanner;

public class CalculoDesconto {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.print("Digite o preço do produto: ");
        double preco = entrada.nextDouble();
        double desconto = 0.1; // 10% de desconto
        double precoFinal = preco * (1 - desconto);

        System.out.println("Preço com desconto: R$" + precoFinal);
        entrada.close();
    }
}
```


Baús de Dados – Armazenando Variáveis



Exemplo: Calculando o Preço de um Produto

```
CalculoDesconto

import java.util.Scanner;

public class CalculoDesconto {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.print("Digite o preço do produto: ");
        double preco = entrada.nextDouble();
        double desconto = 0.1; // 10% de desconto
        double precoFinal = preco * (1 - desconto);

        System.out.println("Preço com desconto: R$" + precoFinal);
        entrada.close();
    }
}
```

- **Scanner entrada:** Lê os dados inseridos pelo usuário.
- **double preco:** Armazena o valor do produto.
- **precoFinal:** Calcula o preço após aplicar o desconto.

Dica: Sempre feche o Scanner com `entrada.close()` para evitar problemas de recursos.

03

Encruzilhadas do Código – Condições

Encruzilhadas do Código – Condições



Condições

Condições permitem que o programa escolha o que fazer com base em uma situação. São como encruzilhadas em uma jornada: você escolhe o caminho a seguir com base na situação, usando **if**, **else if** e **else**.

Encruzilhadas do Código – Condições



Exemplo: Verificando Idade para um Evento

Imagine que você é um guardião e precisa verificar a idade para permitir a entrada em um festival:

```
VerificarIdade

import java.util.Scanner;

public class VerificaIdade {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.print("Digite sua idade: ");
        int idade = entrada.nextInt();

        if (idade >= 18) {
            System.out.println("Entrada permitida!");
        } else {
            System.out.println("Entrada negada. Menor de 18 anos.");
        }
        entrada.close();
    }
}
```

Encruzilhadas do Código – Condições



Exemplo: Verificando Idade para um Evento

```
VerificarIdade

import java.util.Scanner;

public class VerificaIdade {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);

        System.out.print("Digite sua idade: ");
        int idade = entrada.nextInt();

        if (idade >= 18) {
            System.out.println("Entrada permitida!");
        } else {
            System.out.println("Entrada negada. Menor de 18 anos.");
        }
        entrada.close();
    }
}
```

- **if (idade >= 18):** Verifica se a idade é 18 ou mais.
- **else:** Define o que acontece se a condição não for atendida.

Dica: Use else if para verificar múltiplas condições, como diferentes faixas etárias.

O4

Feitiços Repetidos – Laços

Feitiços Repetidos – Laços



Laços de Repetição

Laços são como feitiços que repetem uma ação várias vezes, economizando esforço. Vamos explorar os dois mais comuns, **for** e **while**, e entender suas diferenças.

Para Repetições Definidas (for)

O laço **for** é usado quando você sabe exatamente quantas vezes quer repetir uma ação, como contar os itens em um baú. Ele tem três partes principais: inicialização, condição e atualização.

Feitiços Repetidos – Laços



Exemplo: Listando Itens de um Cardápio

Imagine que você tem uma taverna e quer listar os pratos disponíveis:

```
for = Cardapio

public class Cardapio {
    public static void main(String[] args) {
        String[] itens = {"Pizza", "Hambúrguer", "Salada", "Suco"};

        for (int i = 0; i < itens.length; i++) {
            System.out.println("Item " + (i + 1) + ": " + itens[i]);
        }
    }
}
```

Saída:

```
Cardapio = Saída

Item 1: Pizza
Item 2: Hambúrguer
Item 3: Salada
Item 4: Suco
```

Feitiços Repetidos – Laços



Exemplo: Listando Itens de um Cardápio

```
for = Cardapio

public class Cardapio {
    public static void main(String[] args) {
        String[] itens = {"Pizza", "Hambúrguer", "Salada", "Suco"};

        for (int i = 0; i < itens.length; i++) {
            System.out.println("Item " + (i + 1) + ": " + itens[i]);
        }
    }
}
```

- **int i = 0:** Inicializa um contador i começando em 0.
- **i < itens.length:** Verifica se o contador ainda não chegou ao tamanho do array (4 itens).
- **i++:** Incrementa o contador a cada repetição.

Feitiços Repetidos – Laços



Para Repetições Indefinidas (while)

O laço **while** é usado quando você não sabe ao certo quantas vezes precisará repetir uma ação, mas sabe a condição que deve ser verdadeira para continuar. Ele só verifica uma condição e executa enquanto ela for verdadeira.

Feitiços Repetidos – Laços



Exemplo: Contando Moedas até Atingir um Valor

Imagine que você está contando moedas de ouro até juntar 10 moedas:

```
while = ContaMoedas

public class ContaMoedas {
    public static void main(String[] args) {
        int moedas = 0;
        int objetivo = 10;

        while (moedas < objetivo) {
            moedas++;
            System.out.println("Moeda " + moedas + " coletada!");
        }
        System.out.println("Você coletou " + moedas + " moedas!");
    }
}
```

Saída:

```
ContaMoedas = Saída

Moeda 1 coletada!
Moeda 2 coletada!
...
Moeda 10 coletada!
Você coletou 10 moedas!
```

Feitiços Repetidos – Laços



Exemplo: Contando Moedas até Atingir um Valor

```
while = ContaMoedas

public class ContaMoedas {
    public static void main(String[] args) {
        int moedas = 0;
        int objetivo = 10;

        while (moedas < objetivo) {
            moedas++;
            System.out.println("Moeda " + moedas + " coletada!");
        }
        System.out.println("Você coletou " + moedas + " moedas!");
    }
}
```

- **moedas < objetivo:** Executa o laço enquanto o número de moedas for menor que 10.
- **moedas++:** Adiciona uma moeda a cada repetição.

Feitiços Repetidos – Laços



Diferenças entre for e while

O **for** é mais estruturado, enquanto o **while** dá mais flexibilidade, mas exige cuidado para evitar loops infinitos (quando a condição nunca se torna falsa).

Dica: Sempre verifique se a condição do **while** eventualmente será falsa, ou seu programa pode travar!

05

Encantamentos Reutilizáveis – Criando Funções

Encantamentos Reutilizáveis – Criando Funções



Funções

Funções são como encantamentos que você pode invocar várias vezes para executar tarefas específicas, ajudando a manter seu código organizado e reutilizável.

Elas permitem que você agrupe um conjunto de instruções em um único bloco com um nome próprio, como **calcularMedia**, e as reutilize sempre que necessário, evitando repetir o mesmo código várias vezes.

Para criar uma função, você define seu nome, os parâmetros (dados que ela recebe) e o que ela retorna (se houver), tornando o desenvolvimento mais eficiente e fácil de manter.

Encantamentos Reutilizáveis – Criando Funções



Exemplo: Calculando a Média de Notas

Imagine que você é um mestre de guilda e quer calcular a média de desempenho de um aprendiz:

```
CalcularMedia

public class CalculaMedia {
    public static double calcularMedia(double nota1, double nota2) {
        return (nota1 + nota2) / 2;
    }

    public static void main(String[] args) {
        double nota1 = 7.5;
        double nota2 = 8.0;
        double media = calcularMedia(nota1, nota2);

        System.out.println("A média do aluno é: " + media);
    }
}
```

Encantamentos Reutilizáveis – Criando Funções



Exemplo: Calculando a Média de Notas

```
CalcularMedia

public class CalculaMedia {
    public static double calcularMedia(double nota1, double nota2) {
        return (nota1 + nota2) / 2;
    }

    public static void main(String[] args) {
        double nota1 = 7.5;
        double nota2 = 8.0;
        double media = calcularMedia(nota1, nota2);

        System.out.println("A média do aluno é: " + media);
    }
}
```

- **calcularMedia:** Função que recebe duas notas (nota1 e nota2) como parâmetros e retorna a média.
- **return:** Envia o resultado (a média calculada) de volta para quem chamou a função.
- A função é chamada no **main** passando os valores **nota1** e **nota2**.

Dica: Use nomes claros para funções, como `calcularMedia`, para facilitar a compreensão e deixe os comentários explicando o propósito, se necessário.

Agradecimentos

Obrigada por ler até aqui

Este eBook foi criado com o auxílio de inteligência artificial e diagramado com cuidado por uma pessoa, com o objetivo de demonstrar as habilidades adquiridas durante o Bootcamp TONNIE - Java and AI in Europe no quesito de escrever prompts.

Agradeço à **DIO** pela oportunidade de aprendizado em uma plataforma inovadora, que combina tecnologia e educação de forma prática e acessível.

Um agradecimento especial ao professor **Felipe Aguiar**, cuja dedicação, didática e clareza foram fundamentais para meu crescimento ao longo do curso.

Ferramentas utilizadas: ChatGPT 4.0, Grok 3, Canva



Caroline-Teixeira - Overview

I am a student of Analysis and System Development by Uninter Group. I have interest in back-end, front-end and programming in general. - Caroline-Teixeira

 GitHub <https://github.com/Caroline-Teixeira>



<https://www.linkedin.com/in/caroline-francieli-teixeira-05a56b266/>